Principled Learning for Service Orchestration: A Research Agenda

Syed Izhan Khilji*, Alireza Furutanpey†, Schahram Dustdar*
*Distributed Systems Group (DSG), TU Wien, Vienna, Austria
{i.khilji, dustdar}@dsg.tuwien.ac.at
†Coovally.ai
a.furutanpey@coovally.ai

***Abstract*—Reinforcement learning (RL) has been studied for service orchestration for two decades, yet production orchestrators at the largest cloud providers do not use RL. We argue that this gap is not primarily an engineering shortfall but the predictable outcome of importing standard RL formulations into environments whose structural properties violate the assumptions those formulations rest on. Different clusters and tenant mixtures induce qualitatively different environment dynamics for the same service, ranging from near-deterministic low-entropy regimes to highly non-stationary high-entropy regimes; standard RL algorithms assume a single stationary Markov decision process and orchestration routinely violates this assumption. We contribute a problem-centred taxonomy that organises orchestration tasks by the MDP properties that determine learnability, and an analysis showing that natural orchestration phenomena (telemetry noise, misspecified SLOs, co-tenant interference, regime shift) are structurally analogous to known adversarial-machine-learning attack classes against which standard RL has documented vulnerabilities. We identify three research directions for principled learning-based orchestration: uncertainty-decomposed unsupervised learning, action-space-first MDP formulations, and perturbation-aware training. We realise the first as Principled Surprise-Agnostic Intrinsic Motivation (PSAIM) and report a small empirical programme that supports the mechanism-level claims in a single-node proxy. On a heuristic-breaking distribution shift in which a strong domain heuristic loses its input assumption, the PSAIM realisation outperforms the heuristic on average slowdown and failure rate at every tested severity by maintaining a learned conservative admission policy that limits the damage of corrupted ordering signals.**

***Index Terms*—service orchestration, distributed systems, reinforcement learning, adversarial machine learning, intrinsic motivation, non-stationary environments, research agenda**

I. Introduction

Service orchestration sits at the heart of every modern cloud and edge platform. The orchestrator decides, often hundreds of times per second across millions of containers, how many replicas of a service to run, where to place them, how to route requests among them, and how to allocate finite compute resources under shifting demand. The quality of these decisions translates directly into operational cost, end-user latency, and the probability that a system survives the next traffic spike or the next regional failure. Two decades after machine learning began transforming adjacent fields, including vision, language, and robotics, the question of whether and how learning should drive these decisions remains substantively unsettled. The ambition is clear, the simulation results are abundant, and the production deployments are not.

Reinforcement learning (RL) has been the dominant academic lens. Orchestration decisions are sequential, feedback-driven, and must track a changing environment. These properties map naturally onto the Markov decision process (MDP) abstraction [1], and a substantial body of work has applied RL to auto-scaling [2], cluster scheduling [3], resource management [4], and related tasks [5], [6]. Results reported in simulation are frequently strong.

Production adoption tells a different story. Google's workload manager, Autopilot, deploys both exponentially weighted moving-window recommenders and supervised machine-learning recommenders, but does not use reinforcement learning to drive its scaling decisions [7]. The Borg ecosystem uses priority-based bin-packing and engineered rules [8], [9]. Kubernetes, the most widely deployed open-source orchestrator, defaults to threshold-based scalers. A pattern emerges that is difficult to dismiss: when organisations with the largest clusters, the most telemetry data, and the strongest engineering resources choose how to orchestrate their services, they do not choose RL. The fact that learning is used at all, in the form of Autopilot's supervised recommenders, makes the absence of RL more striking: it is not a rejection of learning, but a rejection of the specific learning paradigm that the academic literature has favoured.

The simplest explanation, that practitioners are conservative and will eventually adopt RL as the methods mature, is unsatisfying because the literature has matured substantially without closing the gap. This paper advances a different explanation. The structural properties of service orchestration, non-stationarity across clusters, partial observability of causal resource-interaction patterns, multi-tenancy, proxy reward signals derived from imperfect SLO telemetry, and action-space constraints imposed by control-plane APIs, violate assumptions that underpin standard RL. Deviations of this kind are not unique to orchestration. What is unique is that they arise together, routinely, and at magnitudes sufficient to compromise convergence, generalisation, and safety simultaneously.

The argument we develop is not that learning is inappropriate for orchestration. It is that the question currently being asked, how can we apply standard RL to orchestration problems more successfully, is less productive than a different one:

what kinds of learning methods are structurally appropriate for orchestration environments?

The contributions of this paper are as follows. We provide a problem-centred taxonomy (Section III) that classifies orchestration tasks by the MDP properties that govern learnability rather than by the RL algorithm applied. Our central analytical contribution (Section IV) is to establish a structural correspondence between natural orchestration phenomena and adversarial-machine-learning attack classes: noisy telemetry maps to observation perturbation, misspecified SLOs to reward poisoning, co-tenant interference to adversarial co-agents, and workload regime shifts to environment poisoning. This correspondence has not been drawn explicitly in the existing literature and reframes RL fragility in orchestration as a predicted consequence of known vulnerabilities in deep RL, rather than as an engineering shortfall. Building on this analysis, we identify three research directions (Section V) for learning-based orchestration that are designed around environment structure rather than against it: uncertainty-decomposed unsupervised learning, action-space-first MDP formulations, and perturbation-aware training. We realise the first direction as Principled Surprise-Agnostic Intrinsic Motivation (PSAIM, Section VI) and report a small empirical programme (Section VII) that tests three specific claims: that natural orchestration-like perturbations degrade learned baselines as the adversarial-ML analysis predicts; that PSAIM exhibits the intended uncertainty-signal separation across hidden entropy regimes; and that the same realisation outperforms a strong domain heuristic in a regime where the heuristic's input assumption is violated. Throughout, we are explicit that the empirical environment is a single-node proxy whose results are mechanism-level evidence rather than deployment evidence; our aim is to clarify and motivate the core ideas, rather than to present a complete system implementation.

II. Background

We briefly recall the reinforcement-learning vocabulary used throughout the paper and introduce an action-space-oriented view of orchestration tasks that anchors the taxonomy of Section III.

A. Reinforcement Learning in Brief

A reinforcement-learning agent interacts with an environment modelled as a Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is a state space, $\mathcal{A}$ an action space, $P(s' \mid s, a)$ a transition kernel, $r(s, a, s')$ a reward function, and $\gamma \in (0, 1)$ a discount factor [1]. The agent's objective is to learn a policy $\pi(a \mid s)$ maximising the expected discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$. Modern methods relevant to orchestration span value-based families (DQN [10], Double-DQN [11]), policy-gradient methods (PPO [12]), actor-critic methods (SAC [13]), and intrinsic-motivation approaches that drive exploration from learning progress rather than extrinsic reward [14], [15]. All share assumptions, a stationary MDP or slow drift, an extrinsic reward signal, and a tractable action space, that are routinely violated in orchestration; this is a recurring theme of the paper.

TABLE I: Action-space structure across orchestration task classes. The structure determines what families of RL methods are applicable and what inductive biases are appropriate.

Task class	Action space	Dominant structure
Horizontal / vertical scaling	Discrete, low-dim. $\{-1, 0, +1\}$ or small integer set	Monotone, feasibility-constrained, reversible
Load balancing & request routing	Continuous, simplex Δ^{n-1}	Sum-to-one, high-frequency decisions, reversible
Resource allocation	Constrained continuous in $\mathbb{R}^{m \times d}_{\geq 0}$	Capacity caps, minimum guarantees, partial reversibility
Service placement & migration	Binary assignment $\{0, 1\}^{n \times m}$	Combinatorial, partially irreversible, cost-asymmetric
SLO-driven control	Mixed discrete / continuous, multi-knob	Heterogeneous, interaction effects, long-horizon

B. Orchestration Tasks from an Action-Space Perspective

The RL literature applied to orchestration typically characterises tasks by their state space and reward structure. We find it more informative to characterise them by their action space. Orchestration actions are not abstract tokens; they are concrete API calls to a control plane, each with known preconditions, costs, and effects. The structure of the action space therefore carries substantial information about what a learning agent can and cannot do, and what formulations are well posed.

Table I summarises the action-space structure for the task classes we examine in detail in Section III. Several observations are immediate. Scaling and placement actions are discrete but subject to monotonicity and feasibility constraints (a replica cannot be removed if none exist, a pod cannot be placed on a node lacking capacity). Load-balancing actions are continuous but live on the probability simplex, with mass that must sum to one. Resource allocation spans a constrained continuous space defined by capacity and minimum-guarantee bounds. Service placement is a combinatorial assignment problem whose size grows with the product of the service count and the node count. SLO-driven control is the most heterogeneous class: actions span multiple discrete and continuous knobs with poorly understood interactions.

The observation that drives this paper's agenda is that while orchestration environments are difficult to model, orchestration *actions* are not. API contracts specify what each action does at a local level; the hard problem is predicting the downstream effect of that action through the environment. This asymmetry, tractable actions operating within intractable dynamics, suggests that the right place to inject prior knowledge is the action space and its structure, rather than the reward function or the environment model. We return to this point in Section V.

III. A Problem-Centred Taxonomy

Having named the action-space structure for each orchestration task class, we now situate each class in a taxonomy organised by the properties that govern whether learning is likely to succeed. Figure 1 provides the overview: each class is characterised by its typical state space, action space, reward structure, observation lag, and risk profile. The taxonomy is not exhaustive, but it captures the classes that account for the large majority of published RL-for-orchestration work.

A. Horizontal and Vertical Scaling

Auto-scaling decides when to add or remove compute instances (horizontal) or adjust per-instance resource limits (vertical) in response to changing workload. The state typically comprises CPU and memory utilisation, request queue depth, and possibly a short-horizon demand forecast. The action space is discrete and low-dimensional.

From a learning perspective, scaling is the most tractable orchestration class. The action space is small, the reward signal (cost minus SLO violations) is well defined, and actions are partially reversible: an unnecessary scale-up wastes money but rarely causes outages. The primary learning difficulty is observation lag. The effect of a scaling action (a new instance becoming ready and absorbing load) is observable only after tens of seconds for containers and minutes for virtual machines, transforming what should be a short-horizon credit-assignment problem into a delayed-reward one that degrades the sample efficiency of standard temporal-difference methods. Rzadca *et al.* [7] describe Autopilot's design as combining moving-window statistics with supervised ML recommenders, deliberately avoiding RL on the grounds of transparency, debuggability, and robustness to workload non-stationarities.

B. Load Balancing and Request Routing

Load balancing distributes incoming requests across a pool of instances. The state includes per-instance load, response times, queue depths, and connection counts. The action is a routing decision per request or a continuous vector of routing weights on the probability simplex.

Decisions are taken per request or per short time window (seconds) and are immediately reversible, making load balancing a comparatively favourable setting for learning. Two issues nonetheless complicate the RL formulation. The first is reward shape: what matters is tail latency (e.g., the 99th-percentile response time), which depends on the cumulative effect of many routing decisions rather than on any single one. The second is that in multi-tenant environments, the environment perceived by the load balancer includes the behaviour of other tenants' workloads, which is uncontrollable and non-stationary. A load balancer trained on one tenant mix may perform poorly when the mix changes, not because the algorithm is wrong but because the MDP itself has changed.

C. Resource Allocation

Resource allocation distributes finite compute resources (CPU cores, memory, GPU time, network bandwidth) among competing tasks or services. Unlike scaling, which adjusts total supply, allocation divides a fixed or slowly changing supply among competing demands, and the action space is multi-dimensional and constrained.

Grandl *et al.* [16] demonstrate that even bin-packing heuristics struggle with multi-resource allocation; the RL formulation inherits this combinatorial difficulty while adding the challenges of credit assignment and exploration. Mao *et al.* [4] showed that deep RL can learn allocation policies that outperform heuristics in simulated small clusters. The simulation, however, was stationary and fully observable. In production, resource-contention patterns shift as services are deployed and retired, tenant workloads fluctuate, and hardware failures change available capacity. A policy converged under one resource regime may perform poorly under another.

D. Service Placement and Migration

Service placement decides which nodes host which service instances; migration extends this to moving instances during execution. Actions are costly and partially irreversible: migrating a stateful service entails downtime, state transfer, and potential data inconsistency. The action space grows combinatorially with the number of services and nodes, and reward depends on inter-service communication, data locality, and node-level availability, which are only partially observable. The high cost of exploration conflicts directly with the need for exploration in RL. Safe RL [17] either constrains exploration to a conservative set or requires a reliable safety model, which is itself difficult in non-stationary environments. Delimitrou and Kozyrakis [18] showed that collaborative filtering over workload profiles can guide placement without trial-and-error exploration, illustrating a recurring pattern: methods that succeed in production exploit domain structure directly rather than discover it through unconstrained exploration.

E. SLO-Driven Control

SLO-driven control is the most general and most difficult class. The agent tunes multiple configuration knobs (thread-pool sizes, cache parameters, connection limits, scaling thresholds) simultaneously to maintain SLO compliance under changing workload. The state space is high-dimensional and partially observable; the action space is combinatorial across knobs; and the reward signal is sparse because SLO violations are infrequent but consequential. Gan *et al.* [19] characterise microservice performance interactions and show that the dependence among knobs requires extensive profiling, undermining the assumption of i.i.d. exploration.

F. Cross-Cutting Observations

Three patterns emerge. First, difficulty for RL increases along two axes simultaneously: MDP complexity (action-space size, partial observability, reward sparsity) and environment non-stationarity (workload drift, tenant churn, hardware changes). Standard RL algorithms are designed to handle one of these challenges at a time. Handling both together remains an open problem. Second, the classes where RL has produced

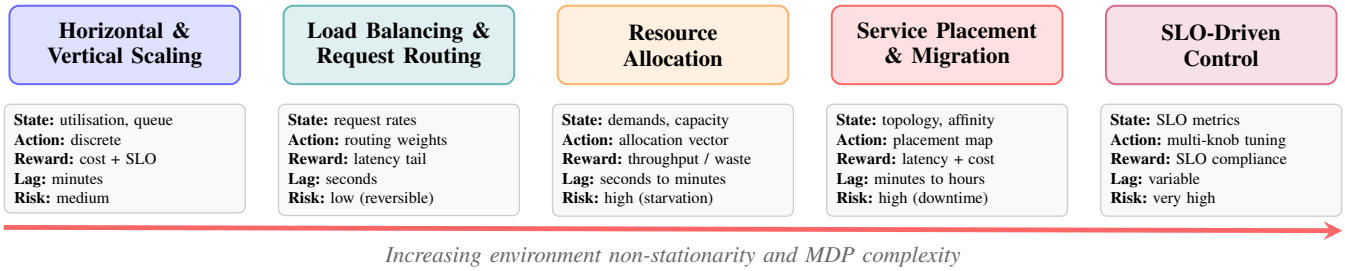

Fig. 1: A problem-centred taxonomy of RL for service orchestration. Each class is characterised by its typical state and action spaces, reward structure, observation lag, and risk profile. Difficulty for learning increases from left to right as non-stationarity, action irreversibility, and MDP complexity grow.

the most credible results (load balancing, simple scaling) are precisely the classes where conventional heuristics [20], [21] already perform well, and the incremental benefit rarely justifies the operational burden of a learned component. Third, as noted in Section II-B, the action space is often far more tractable than the environment dynamics, but standard RL formulations treat them symmetrically: the same Q-function approximates the value of every action, with no structural prior beyond what is learned from data.

IV. Why Naive RL Import Falls Short

The taxonomy identifies the structural properties that make orchestration hard for RL. This section tightens the argument by examining why these properties lead to predictable failure modes, and by showing that many of those modes are structurally analogous to adversarial-machine-learning attack classes against which standard RL algorithms have documented vulnerabilities.

A. Failure Modes as Expected Outcomes

a) Non-stationarity as the default condition.: Convergence guarantees for both tabular and function-approximation RL [22], [10] assume a stationary MDP or slow drift that the learning rate can track. Padakandla [23] surveys RL methods for dynamically varying environments but notes that most assume the environment cycles through a finite set of known modes. Production orchestration does not cycle; it drifts, jumps, and occasionally undergoes phase transitions when, for example, a viral event pushes demand outside the capacity model. A policy converged under one regime can take precisely the wrong actions under another, because the values it encodes were correct under assumptions that no longer hold.

b) Sim-to-real gap.: Most RL-for-orchestration work is trained and evaluated entirely in simulation. The simulators in the literature [4], [3], [24] are designed to expose the gross structure of orchestration (queueing, multi-resource packing, job arrival processes) and have produced informative results within their scope. What they do not capture, by design, are the features that dominate observed performance in production: tail-latency dynamics under contention, interference from co-located services, network micro-bursts, OS-level scheduling jitter, and garbage-collection pauses. Robotics, where physics simulators are far more mature, still documents systematic sim-to-real degradation [25]; in orchestration, where simulators are simpler relative to the underlying systems they model, the gap is correspondingly larger. A policy that achieves strong simulated performance therefore licenses no quantitative claim about deployment performance, even when the simulator is qualitatively faithful to the task structure.

c) Reward specification.: The true orchestration objective is a context-dependent trade-off among cost, latency, reliability, and cross-tenant fairness. No scalar reward captures this trade-off without simplification. RL agents optimise whatever reward they are given, which may diverge from operator intent: a cost-minimising agent may consolidate aggressively and increase tail latency, an SLO-compliance agent may over-provision. These are not algorithmic failures; they are failures of reward specification, inherent to the approach.

d) Exploration risk.: Exploration in orchestration means taking actions whose consequences are uncertain, sometimes including outages or data loss. Safe RL [17] either constrains exploration (limiting what can be learned) or requires a safety model (hard to obtain in non-stationary settings).

e) Cluster heterogeneity.: A subtlety often overlooked is that different clusters, even within one organisation, induce fundamentally different environments for the same task. Public traces of production workloads document this directly. Reiss *et al.* [26] characterise the Google compute cluster trace and find that workload heterogeneity (in resource demand, duration, and tenant constraints) is the most prominent statistical feature of the data, sufficient to defeat slot-based and core-based scheduling assumptions that work on more uniform workloads. Cortez *et al.* [27] report a similar pattern in the Microsoft Azure VM workload, with first-party and third-party tenants exhibiting qualitatively different distributions of lifetime, deployment size, and resource consumption. A cluster running latency-sensitive web services has low-entropy, near-deterministic dynamics where conservative and predictable policies perform well. A cluster running bursty batch workloads has high-entropy, stochastic dynamics where the appropriate policy must tolerate uncertainty and explore when warranted. An RL agent trained on one workload distribution and deployed against another faces not a degree-of-difficulty change but a change in the underlying MDP, against which

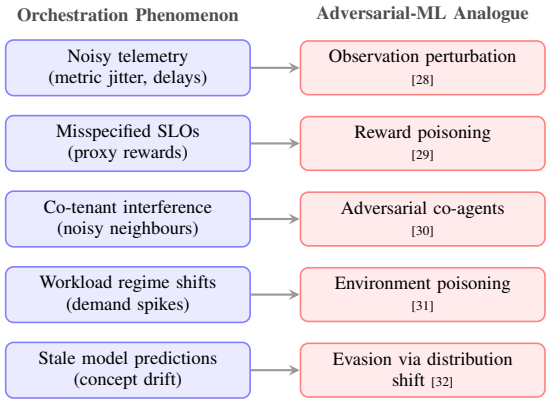

Fig. 2: Mapping between natural orchestration phenomena (left) and their structural analogues in the adversarial-machine-learning literature (right). The orchestration environment produces these perturbations incidentally, but their effect on RL policies is the same as if they were injected deliberately.

standard convergence arguments make no claim.

B. An Adversarial-Machine-Learning View

A sharper analysis is possible by drawing on adversarial machine learning, which characterises the vulnerability of learned models to deliberate or incidental perturbations. Many phenomena that occur naturally in orchestration are structurally analogous to known attack classes. We do not claim that orchestration environments are adversarial by intent; we claim they are adversarial in effect. Figure 2 maps each natural orchestration phenomenon to its adversarial-ML counterpart.

The implications go beyond rhetorical analogy. For each row of Figure 2 the adversarial-ML literature establishes a fragility result whose conditions are then routinely satisfied in production orchestration, even though no adversary is present. We state each correspondence explicitly so that it generates a prediction the experiments in Section VII can test.

a) Observation perturbation → noisy telemetry.: Production telemetry is collected at sampling intervals on the order of seconds to tens of seconds, aggregated with delay, and per-counter jitter from kernel scheduling, container quotas, and metric pipelines is routine. The orchestration agent therefore observes a perturbed signal whose magnitude relative to the underlying state is qualitatively comparable to the perturbation budgets explored in adversarial RL: Huang *et al.* [28] and Zhang *et al.* [32] demonstrate that bounded observation perturbations of comparable magnitude suffice to induce sharp degradation in policies trained with DQN, A3C, and TRPO. The prediction that follows, and that Section VII-B tests, is that learned allocators degrade under realistic Gaussian observation noise without any adversarial intent.

b) Reward poisoning → misspecified SLOs.: Rakhsha *et al.* [29] establish that small but systematic reward perturbations are sufficient to steer an RL agent toward an arbitrary target policy. The condition that makes this attack effective in their work is not large magnitude but consistency: the perturbation must point in a coherent direction over time. Production SLO telemetry satisfies exactly this condition. Latency measured at the load balancer is systematically lower than what end users experience; cost computed from pricing models is systematically discretised against the true marginal cost; threshold-based reward signals with hysteresis are systematically biased against the boundary between healthy and unhealthy regions. The prediction is that small, biased, time-correlated reward perturbations will degrade learned allocators sharply, even when comparable random reward noise would not.

c) Adversarial co-agents → co-tenant interference (qualified).: The mapping from adversarial co-agents [30] to co-tenant interference is the weakest of the four and we phrase it accordingly. Adversarial co-agents are policies optimised to exploit a learned policy. Co-tenant workloads in production are not optimised against the orchestrator: they are the resource demands of unrelated services that happen to share infrastructure. The structural correspondence is therefore not at the level of optimisation, but at the level of *distributional shift in co-agent behaviour*: the co-agent policy at deployment is drawn from a different distribution than the one assumed during training, and Gleave *et al.* show that this alone is sufficient to induce policy collapse. The prediction is that learned allocators trained without exposure to a representative tenant mixture will degrade sharply when deployed against an unfamiliar one, even when the unfamiliar tenant exerts no strategic pressure. The matched-load control in Section VII-B is designed to separate this distributional-shift effect from a pure load increase.

d) Environment poisoning and distribution shift → workload regime shifts and concept drift.: A marketing event, a software release, a hardware degradation, or a tenant onboarding routinely changes the underlying transition kernel of an orchestration environment without changing the agent's representation of it. This is the operational analogue of two distinct adversarial-ML attack classes. Behzadan and Munir [31] establish that perturbations to the environment dynamics during training produce policies whose behaviour at deployment is incoherent; the regime shift maps onto this directly. Concept drift in operating conditions is then equivalent to evasion via distribution shift in the supervised sense [32]: the deployment-time observations come from a distribution increasingly distant from the training-time distribution, and the policy's learned values become decreasingly correct. We accordingly predict, and Section VII-B confirms in proxy form, that allocators trained on one workload regime degrade when evaluated on another in proportion to the distance between the two.

These four predictions structure the empirical programme in Section VII. The novelty we claim for the analysis is not in any individual correspondence but in the cumulative observation that production orchestration assembles, by accident, the conditions under which deep RL is known to fail by adversarial design. Existing safe-RL and robust-RL literature treats these vulnerabilities as deliberate attack surfaces; we treat them as the default operating conditions of an entire application domain.

V. Research Directions

The preceding analysis argues that the problem is not with learning as a paradigm but with the specific formulations that have been imported from other domains. The Autopilot example introduced in Section I is suggestive in this respect: Google's production answer to scaling has been to deploy supervised machine-learning recommenders that produce a prediction, while leaving the act of choosing a configuration to a deterministic moving-window controller [7]. The structural choice is not learning-versus-no-learning; it is learning at a remove from the policy itself. The directions we identify do not advocate replicating that specific architecture, but they are sympathetic to the underlying intuition: the act of *deciding* should be guarded by structural properties (uncertainty awareness, action-space typing, perturbation tolerance) that the act of *predicting* need not provide.

The three directions identified below are derived from the failure modes of Section IV, not chosen at random. Section IV-A traces fragility to three orthogonal structural mismatches: (i) RL assumes a single stationary MDP, while orchestration exposes the same agent to clusters whose transition kernels differ qualitatively; (ii) RL treats the action space as an interchangeable index set, while orchestration actions carry structural semantics (capacity, monotonicity, irreversibility) that the function approximator has to recover from data; and (iii) RL is evaluated under benign training distributions, while orchestration deploys learned policies into environments whose perturbation profile is structurally analogous to adversarial-ML attacks (Section IV-B). Each direction targets one of these mismatches. D1 confronts (i) by giving the agent an internal representation of which uncertainty is reducible by learning and which is not, so a single policy can behave coherently across regimes. D2 confronts (ii) by elevating the action-space typology of Table I to the primary locus of inductive bias, rather than recovering it implicitly through function approximation. D3 confronts (iii) by making perturbation evaluation a default rather than an optional check, so that agents whose nominal performance does not survive realistic perturbation are filtered out before deployment. These are not the only directions consistent with the failure analysis: alternatives such as full robust-MDP minimax training (Section VIII), offline RL from production traces, or model-based RL with calibrated dynamics models are all defensible responses to subsets of the same diagnoses. We argue for the directions below specifically because each isolates one mismatch cleanly enough to support empirical testing, and because their intersection covers the failure modes the field has empirically encountered.

We now identify the three directions in detail. They are not mutually exclusive; instances of any two can reasonably coexist in a single system. They also do not address the taxonomy of Section III uniformly. D1 (uncertainty-decomposed learning) most directly addresses the task classes where regime diversity dominates, namely scaling and resource allocation, and where the same agent must operate across clusters with different entropy profiles. D2 (action-space-first formulations) most directly addresses the classes where action-space combinatorics dominate, namely service placement and SLO control, and where the typology of Table I is most informative. D3 (perturbation-aware training) cuts across all classes because telemetry imperfections, reward proxies, and co-tenant effects appear in every orchestration task; the experiments in Section VII test it on resource allocation but the conclusions transfer.

A. D1: Uncertainty-Decomposed Unsupervised Learning

The first direction addresses the regime-diversity problem identified in Section IV-A. Standard RL assumes one MDP and learns one policy; orchestration exposes agents to a mixture of low-entropy and high-entropy regimes that require qualitatively different behaviour. In a low-entropy regime where dynamics are near-deterministic but poorly characterised, an agent should seek informative transitions; in a high-entropy regime where dynamics are well characterised but inherently stochastic, it should avoid chasing unpredictable fluctuations.

A natural way to separate these cases is to distinguish the two kinds of uncertainty that produce surprise. Epistemic uncertainty reflects what the agent does not yet know and learning can reduce. Aleatoric uncertainty reflects what is inherently unpredictable and no amount of data will resolve. The standard curiosity-based intrinsic rewards [14], [15], [33], [34], [35] do not make this distinction and therefore fail in the "noisy TV" setting [36], where the agent is drawn indefinitely to irreducible noise. Meta-controllers that switch among exploration strategies [37] address the symptom by mode-switching, but do not provide a principled decomposition.

A research direction, then, is to build unsupervised learning signals that explicitly decompose total uncertainty into epistemic and aleatoric components, using the decomposition to drive a single adaptive reward rather than a portfolio of exploration strategies. The Bayesian view of this decomposition is established [38], [39]; what remains open is a computationally practical realisation that is compatible with deep value-based RL and that behaves coherently under the kinds of non-stationarity orchestration exhibits. Section VI sketches one instantiation.

B. D2: Action-Space-First MDP Formulations

The second direction addresses the asymmetry observed in Section II-B and Section III-F: orchestration actions are well-typed API calls whose local behaviour is known by contract, while orchestration dynamics are not. Standard RL formulations treat both sides symmetrically, approximating $Q(s, a)$ without structural prior beyond what can be learned from data, and leaving the action-space structure to be recovered implicitly through function approximation.

An action-space-first formulation treats the action space and its constraints as part of the problem specification rather than part of the unknown to be learned. Concrete versions of this direction include: (i) policies that decompose action selection into a structured sub-policy per action type, so that scaling,

placement, and routing each use an architecture matched to their action-space geometry (discrete and monotone, combinatorial assignment, continuous simplex respectively); (ii) constrained action representations that make infeasible actions unrepresentable rather than penalised, so that exploration cannot violate capacity or feasibility bounds by construction; and (iii) hierarchical formulations in which a high-level agent selects the action type (which API call) and a low-level agent selects its parameters, aligning the hierarchy with the action-space decomposition shown in Table I.

The individual ingredients of D2 have precedents in the broader RL literature: constrained action representations have been studied for safe RL, hierarchical action selection has been studied for long-horizon control, and structured policies have been used in domains with discrete combinatorial actions. The contribution we identify is a different one: to elevate the action space from incidental detail to the primary locus of inductive bias for orchestration RL. Concretely, this means that researchers proposing new orchestration RL methods should justify their choice of policy parameterisation against the action-space typology of Table I, rather than defaulting to architectures imported from Atari benchmarks where action structure is uniform across tasks. The current norm reverses these priorities: the bulk of engineering effort goes into state representation and reward shaping, while action-space structure is recovered implicitly through function approximation. We argue this is the wrong allocation of inductive bias.

An empirical test of D2 is not provided here. The instantiation in Section VI and the experiments in Section VII address D1 (uncertainty decomposition); the stress experiments in Section VII-B address D3 (perturbation-aware evaluation); D2 is identified as a research direction but its concrete proposals are not evaluated here. We regard D2 as the direction most in need of empirical work and the most natural target for a follow-up study, particularly one that compares an action-space-typed policy against a flat-action baseline on placement or scheduling tasks where the action-space structure of Table I is non-trivial.

C. D3: Perturbation-Aware Training

The third direction follows from Section IV-B. If production orchestration exposes agents to perturbations that are structurally analogous to adversarial attacks, then methods designed without this analogy in mind will fail, regardless of how much benign training data they consume.

Perturbation-aware training treats noise in telemetry, bias in reward proxies, and drift in co-agent behaviour as first-class design variables rather than as artefacts to be averaged away. Concrete elements include robust training objectives over bounded observation perturbations in the spirit of Zhang *et al.* [32], reward-proxy audits that characterise the sensitivity of the learned policy to realistic reward corruption, and co-agent-aware training in which tenant-mixture distributions are varied during training to approximate the multi-agent exposure a deployed policy will face. Safe RL [17] is relevant but insufficient: safety is one axis of perturbation awareness among several, and most safe-RL methods assume that the adversary is the agent itself, not the environment.

The broader claim of this direction is that RL for orchestration should be evaluated under stress by default, with stress tests calibrated to match the magnitudes of perturbation that realistic production telemetry imposes. Agents that do not degrade gracefully under such stress cannot be trusted regardless of their nominal performance.

D. Per-Class Predictions

The taxonomy of Section III is most useful when it constrains expectations. We close this section by stating, for each task class, what we expect the directions to deliver and what we do not. These predictions are deliberately committal rather than hedged: they are the claims the community can hold us, and one another, accountable for.

Scaling. The action space is small and reversible, the regime variation across clusters is real but bounded, and the dominant difficulty is observation lag. We expect D1 to produce a meaningful improvement here: an agent that knows the difference between epistemic uncertainty (a workload it has not seen) and aleatoric uncertainty (load it has seen and learned is noisy) should scale conservatively in the former case and decisively in the latter. We do not expect D2 to be load-bearing for scaling, because the action space is already nearly minimal.

Load balancing and request routing. The action space lives on the simplex, the feedback is fast, and reversibility is high. We expect D3 to be the dominant direction here: small perturbations to the input feature space (which instance is loaded, which is reachable, which is failing slowly) will silently corrupt routing decisions in production, and benign-training agents will not detect this. We do not expect D1 to add much over a well-trained policy-gradient agent in this class.

Resource allocation. This is the class for which we have the most empirical evidence (Section VII) and where all three directions are plausibly load-bearing. D1 manages regime variation across job mixes, D2 typing matters because allocation is multi-resource and constrained, D3 is critical because reward proxies (completion time, utilisation) are exactly the inputs that get corrupted under load.

Service placement and migration. Combinatorial action space, partial irreversibility, expensive exploration. We expect D2 to be the dominant direction: structured action representations and hierarchical decomposition of the placement problem will outperform flat function approximation by a margin that grows with the number of services. D3 is also relevant because placement is exposed to severe distribution shift across clusters.

SLO-driven control. The most heterogeneous and least understood class. We do not predict any single direction to dominate. What we do predict is that any RL formulation that ignores all three directions will produce policies that are uninterpretable, fragile, and operationally unacceptable, regardless of nominal SLO compliance in simulation.

These predictions are intended to be falsifiable. A study showing that D2 is unnecessary for placement, or that D3

is irrelevant for load balancing, would update the framework rather than overturn it; the value of stating the predictions explicitly is to make subsequent work cumulative rather than parallel.

VI. PSAIM: An Operational Realisation of D1

To show that the directions of Section V are not vacuous, we describe one mechanism realisation of D1 in sufficient detail to make its behaviour checkable both analytically and empirically. We call the realisation Principled Surprise-Agnostic Intrinsic Motivation (PSAIM). PSAIM is presented at the level of mechanism rather than as a fully specified system: the core uncertainty-decomposition is given, the architecture is specified, and the empirical signature is reported in Section VII, but supporting machinery for formal guarantees (calibration of ensemble heads, bounded aleatoric estimators with concentration arguments) is left for separate development.

This section describes the full mechanism. The empirical evaluation in Section VII uses a deliberately simplified configuration that retains the core uncertainty decomposition (Equation (2)) and the gated intrinsic reward (Equation (4)), and replaces the calibrated quantile-regression heads and probe-augmented aleatoric estimators with simpler scalar Q-heads and raw within-head variance. This separation is intentional: the empirical findings should be read as a test of the mechanism, not of the full architecture, and the additional machinery described below is required only when the simpler configuration's accuracy bounds become limiting.

A. Idealised Objective

Let $\theta \sim p(\theta \mid D)$ parameterise a dynamics model $p_\theta(\cdot \mid s, a)$ given experience D. The idealised exploration–control objective is

$$U_{\text{PSAIM}}(s,a) \;=\; \underbrace{I(\theta;\, S' \mid s,a,D)}_{\text{epistemic info gain}} \;-\; \lambda\, \underbrace{\mathbb{E}_\theta\big[H(S' \mid s,a,\theta)\big]}_{\text{aleatoric entropy}}, \tag{1}$$

where I denotes conditional mutual information and H denotes Shannon entropy. The first term rewards actions that reduce uncertainty about the dynamics; the second penalises actions whose outcomes are inherently unpredictable. Which term dominates depends on the local entropy regime: in a low-entropy but poorly characterised region, the first term is large and the agent explores; in a high-entropy but well-characterised region, the second term dominates and the agent avoids noise. The behaviour switches automatically without regime labelling.

The ideal objective is analogous in spirit, though not in formalism, to information-directed sampling [40], which trades off information gain against regret in the bandit and tabular settings. PSAIM targets the deep-RL setting with function approximation, where the Bayesian posterior over dynamics models is intractable and a model-free surrogate is required.

B. Model-Free Surrogate via the Law of Total Variance

We replace Equation (1) with a surrogate that uses an ensemble of value-function heads and the law of total variance. Consider K statistics heads $\{E_k\}_{k=1}^{K}$, each estimating a return distribution, for instance through quantile regression [41], [42]. Under a reference policy π_{ref}, define a policy-consistent continuation $g_k(s) = \mathbb{E}_{a\sim\pi_{\text{ref}}}[E_k(s,a)]$ and a return-space one-step variable $Y_k = \gamma\, g_k(S')$. The law of total variance decomposes the variance of Y_k into

$$\mathrm{Var}(Y_k) = \underbrace{\mathrm{Var}_k\big(\mathbb{E}[Y_k \mid s,a]\big)}_{V_{\text{epi}}:\text{epistemic}} + \underbrace{\mathbb{E}_k\big(\mathrm{Var}[Y_k \mid s,a]\big)}_{V_{\text{ale}}:\text{aleatoric}}, \tag{2}$$

where Var_k and $\mathbb{E}_k$ denote variance and expectation taken over the ensemble index k, while the inner $\mathbb{E}[Y_k \mid s,a]$ and $\mathrm{Var}[Y_k \mid s,a]$ are taken over the next-state distribution conditional on (s,a). The first term V_{epi} measures how much the heads disagree about the conditional mean of Y_k: this is large when the ensemble has not converged on a single function, which is the signature of model uncertainty. The second term V_{ale} measures the average within-head conditional variance: this is large when next-state outcomes are intrinsically spread, which is the signature of irreducible noise. No Bayesian posterior semantics are assumed; V_{epi} and V_{ale} are operational proxies computed from the ensemble.

We note one limitation explicitly. The operational V_{epi} measured from a finite ensemble of value-function heads conflates two distinct sources of variance: genuine epistemic disagreement about regions the agent has not characterised, and head-wise variance induced by fitting noisier targets in high-aleatoric regions of the state space. This is a known concern about ensemble-based epistemic estimation in deep RL more broadly: a finite ensemble cannot perfectly separate the variance it would have asymptotically (true epistemic) from the variance it inherits from the data it is fitting. The decomposition in Equation (2) is therefore exact as an algebraic statement about the random variable Y_k, but the quantity we measure for V_{epi} is an upper bound on the true epistemic uncertainty. The gate in Section VI-C is designed in part to mitigate the operational consequences of this conflation, by treating regions where high V_{epi} is accompanied by high V_{ale} as noise-dominated rather than information-rich; a more refined decomposition would require either a calibrated probabilistic ensemble or a separate head-noise estimator, both of which we leave for future work. Ensemble-based epistemic signals have nonetheless a long record in RL exploration [43], [44]; the contribution of the present construction is the explicit decomposition and its integration into a single reward, with the conflation acknowledged rather than ignored.

C. Intrinsic Reward and Short-Horizon Gate

High aleatoric uncertainty should not automatically penalise an action if that action immediately opens a learnable region. PSAIM includes a short-horizon epistemic gate that looks h_g steps ahead under a probe policy and discounts the aleatoric

penalty by the anticipated epistemic value:

$$\widetilde{V}_{\text{epi}}^{(h_g)}(s,a) = \mathbb{E}\Bigg[\sum_{j=1}^{h_g} \beta^{j-1}\, V_{\text{epi}}\big(S_{t+j}, \pi_{\text{probe}}(S_{t+j})\big) \;\Bigg|\; S_t = s,\ A_t = a\Bigg], \tag{3}$$

where $\beta \in (0,1]$ is a gate discount. Defining the gated aleatoric excess $\Delta(s,a) = \big[V_{\text{ale}}(s,a) - \alpha\, \widetilde{V}_{\text{epi}}^{(h_g)}(s,a)\big]_+$, the intrinsic reward is

$$r_{\text{int}}(s,a,s') \;=\; V_{\text{epi}}(s,a) \;-\; \lambda \log\!\left(1 + \frac{\Delta(s,a)}{\sigma_0^2}\right), \tag{4}$$

with $\sigma_0 > 0$ a baseline scale and $\alpha, \lambda \geq 0$ mixing weights. The logarithmic form keeps the penalty bounded and produces smooth gradients. The gate implements a quantitative version of the "noisy TV" guardrail: when the near future is learnable, the aleatoric penalty is softened; otherwise, it remains strong.

D. Architecture and Learning Rules

Figure 3 summarises the flow of information in a PSAIM agent. The statistics ensemble and the reference policy used for continuation are held fixed over blocks of training, decoupling the measurement of uncertainty from the policy being optimised. Within each block, r_{int} is a fixed function of (s,a,s') and the control critics are trained by standard Double-DQN [11] with r_{int} in place of an extrinsic reward. The statistics heads are trained by distributional policy evaluation against π_{ref}. Between blocks, the reference policy and the ensemble snapshot are refreshed to reflect accumulated experience.

We make no formal convergence claims in this paper. PSAIM is offered as one way to realise D1 with well-understood components; stronger guarantees (bounded concentration, contractive Bellman operators under stationary rewards) require machinery that we do not develop here. The empirical evaluation in Section VII-A uses a deliberately simplified configuration of the architecture described above; the deviations are stated explicitly there.

E. Relevance to Orchestration

PSAIM is designed around three properties, each motivated by a structural feature of orchestration environments. The surprise-agnostic switch is designed to allow a single agent to operate across the entropy regimes that different clusters induce, without manual regime detection (Section V-A). The epistemic gate is designed to provide a principled response to noisy telemetry, by using anticipated short-horizon learnability to soften the aleatoric penalty when the near future is informative. The block-wise freezing of reward-defining objects is designed to produce piecewise-stationary learning, sidestepping the non-stationarity that invalidates convergence arguments for globally trained RL in orchestration.

The empirical section (Section VII) tests how much of this design intent is realised at the proxy scale. Of the three, only the surprise-agnostic switch produces a behavioural separation that the proxy is large enough to exhibit (Section VII-C); the gate and the freezing protocol are not load-bearing in the proxy at any horizon we have tested (Section VII-E), and we are explicit that this is consistent with their design targets being absent from the proxy rather than being evidence that they are needed. None of these properties constitutes a solution to orchestration. Together, they characterise an agent whose assumptions are less violated by orchestration environments than those of the standard RL baseline; whether that translates into deployment-relevant performance is an empirical question for environments richer than the present proxy.

VII. Empirical Findings

The viability of the directions in Section V is ultimately an empirical question. We report a small experimental programme that addresses three specific claims made in this paper. The first is that natural orchestration-like perturbations degrade learned allocation baselines in the manner predicted by the adversarial-ML analysis of Section IV-B. The second is that the PSAIM realisation of Section VI, as a mechanism realisation of D1, exhibits the intended uncertainty-signal separation across hidden entropy regimes. The third is that the same PSAIM realisation produces a downstream behaviour, conservative admission under unreliable input features, that yields a measurable performance advantage over a strong domain heuristic in a regime where the heuristic's input assumption is violated. We use the term *proxy* throughout: the environment is a single-node, single-agent allocator that captures core orchestration phenomena in simplified form. It is not a substitute for production orchestration.

A. Setup

a) Environment.: We construct a single-node resource allocator with two resource dimensions (CPU cores and memory) that receives a stream of heterogeneous tasks. At each decision step the agent observes current utilisation, the queue of pending tasks, and (under stress conditions) imperfect telemetry for these quantities. The action space is discrete: allocate the head-of-queue task, defer it, or reject it. The episodic reward is a function of completion time, utilisation, and task-failure rate.

b) Baselines.: We compare four learned baselines against a strong domain heuristic:

- **SJF heuristic:** shortest-job-first with best-fit allocation; non-learned.
- **PG proxy:** a policy-gradient resource allocator following the formulation of Mao *et al.* [4].
- **DQN:** a standard Double-DQN allocator [11], [10].
- **DQN+RND:** DQN augmented with Random Network Distillation curiosity [15], included as a stronger intrinsic-motivation baseline so that PSAIM is compared against a curiosity-driven method, not only against bare DQN.
- **Simplified PSAIM:** the sketch of Section VI with $K{=}5$ scalar-valued Q-heads in place of full quantile-regression heads, raw within-head variance in place of probe-augmented aleatoric estimators, and an epistemic-gate horizon $h_g{=}3$.

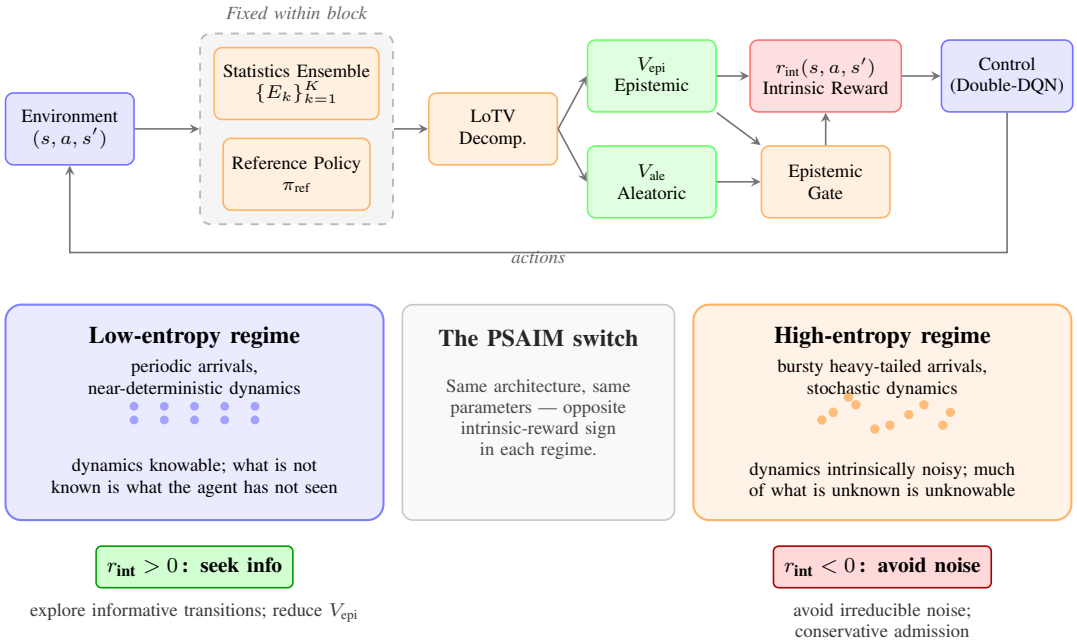

Fig. 3: PSAIM at a glance. **Top:** information flow. A statistics ensemble $\{E_k\}_{k=1}^{K}$ and a reference policy π_{ref} are frozen within each block, decoupling uncertainty measurement from control. The law of total variance produces epistemic (V_{epi}) and aleatoric (V_{ale}) components; the epistemic gate softens the aleatoric penalty when near-future learnability is high; the resulting intrinsic reward r_{int} drives a Double-DQN controller. **Bottom:** the behavioural switch. With identical architecture and parameters, r_{int} takes opposite signs across hidden entropy regimes—positive when epistemic value dominates (information-seeking exploration that drives down V_{epi}), negative when irreducible aleatoric noise dominates (conservative admission). Surprise-agnosticism is a property of the reward structure itself, not of an external switching mechanism.

c) Evaluation protocol.: All reported numbers are means with 95% bootstrap confidence intervals over 10 seeds. For Experiment 1 we evaluate at three severity levels (low, medium, high) per stress scenario and report degradation relative to the matched clean baseline. For Experiment 2 we use a held-out evaluation episode in which the agent is exposed to alternating low- and high-entropy workload blocks without being told which regime is active.

d) Configuration details.: All learned agents are trained for 100 episodes of 96 decisions each, with a 96-step warm-up before gradient updates begin and gradient updates every 4 environment steps thereafter; the target network is updated every 24 steps. Q-networks share a 128–128 ReLU MLP trunk with 5 ensemble heads and bootstrap-mask probability 0.8 per head. The optimiser is Adam with learning rate 3×10^{-4}, batch size 64, replay buffer capacity 20 000, gradient clipping at norm 1.0, discount factor $\gamma = 0.98$, and an ϵ-greedy schedule from 1.0 to 0.05 decayed over 2 600 steps with evaluation $\epsilon = 0.01$. PSAIM-specific hyperparameters: intrinsic reward scale 0.001, aleatoric weight $\lambda = 3.0$, gate threshold $\alpha = 0.16$, normaliser $\sigma_0^2 = 0.05$, gate horizon $h_g = 3$, aleatoric EMA decay 0.95, epistemic prior floor 0.35. Workload generators: periodic arrivals use a deterministic Poisson process; bursty heavy-tailed arrivals use a Pareto inter-arrival distribution. Stress severities (low/medium/high) parameterise observation-noise standard deviation, reward-bias magnitude, and duration-misreport bias relative to ground truth; complete parametric specifications are released with the code. Training and evaluation use 10 seeds throughout and run on CPU.

e) Code and data availability.: The PSAIM realisation, the proxy environment, and the evaluation scripts will be released publicly at [REPOSITORY URL TO BE FILLED] on publication, with full hyperparameter and workload-generator specifications.

B. Stress Fragility of Learned Baselines

We evaluate four stress scenarios, each calibrated to the adversarial analogue in Figure 2: **(a) Observation noise** adds Gaussian noise to utilisation metrics with standard deviation chosen to match realistic telemetry jitter; **(b) Reward corruption** delays the completion-time signal and applies a small systematic bias; **(c) Distribution shift** trains on a periodic workload and evaluates on a bursty heavy-tailed workload; **(d) Co-tenant interference** injects a second, uncontrolled stochastic workload on the same node during evaluation. We additionally include a **co-tenant matched-load control**, in which a deterministic constant load equal to the expected co-tenant demand is added. The control rules out the trivial reading that "adding load hurts" by isolating the contribution of co-tenant *stochasticity* as opposed to mere load.

Figure 4 shows degradation, defined as the percentage increase in average slowdown relative to the clean condition,

as a function of stress severity for the three policies whose performance is most sensitive to perturbation. Both learned methods degrade substantially across all four scenarios at high severity, with degradations at least 40% above the clean baseline in every case; the SJF heuristic is included for reference and is the least sensitive across the board.

The high-severity degradations are substantial across all four scenarios (PG proxy: 41–135 % range; DQN: 79–350 % range), with all bootstrap CIs strictly above zero. The largest degradations for both methods come from reward corruption, consistent with the prediction in Section IV-B that small but systematic reward perturbations are particularly effective at steering RL policies. The SJF heuristic is included for reference and registers exactly 0% degradation under reward corruption (it does not consume a learned reward), with smaller-than-learned-method degradations under the other three scenarios.

a) Stochasticity matters, not just additional load.: The matched-load control isolates the role of co-tenant stochasticity. At high severity, DQN degrades by 51.5 % under deterministic matched load and 79.4 % under stochastic interference; the PG proxy shows 26.6 % versus 51.7 % at the same comparison. The gap is consistent with the distributional-shift reading of Section IV-B: unpredictability is harder for a trained policy to absorb than equivalent predictable load. A single deterministic control is suggestive rather than definitive; a multi-control study across deterministic intensities is identified as natural follow-up.

b) Reward-corruption robustness is severity-dependent.: We further examined whether PSAIM's intrinsic-reward signal partially decouples the agent from corrupted extrinsic reward. At low severity, simplified PSAIM has lower stressed slowdown than DQN by $-2.07\,[-4.13, -0.63]$ and lower degradation by $-59.7\,[-117.1, -17.8]\,\%$, both with CIs entirely below zero. At high severity the picture reverses: PSAIM degrades comparably to DQN ($+290.7\,\%$ versus $+287.4\,\%$) with paired difference not separated from zero. Mechanism-honestly: at low severity the intrinsic-reward signal carries useful information; at high severity the corrupted extrinsic reward dominates the Bellman target regardless of intrinsic-reward magnitude, and the present realisation has no mechanism to detect or downweight reward-channel corruption. A reward-trust mechanism (for example, reward prediction ensembles or downweighting of extrinsic reward when corruption indicators are detected) would extend the realisation in this direction and is identified as concrete follow-up.

c) Scope of the stress evaluation.: The four scenarios reproduce the qualitative failure pattern predicted by the adversarial-ML mapping in a controlled proxy. This grounds the claim that learned baselines degrade under realistic perturbations even in a benign environment, but does not quantify what those degradations would look like in production, where perturbation magnitudes and correlations differ from the proxy.

C. PSAIM Signal Behaviour Across Hidden Regimes

We now examine whether the PSAIM instantiation produces the qualitative signal behaviour predicted in Section VI. The agent is evaluated on a held-out episode in which the environment alternates between a low-entropy regime (periodic arrivals, homogeneous demands) and a high-entropy regime (heavy-tailed arrivals, heterogeneous demands), without exposing the regime label to the agent. Figure 5 shows the within-episode trajectories of V_{epi}, V_{ale}, and r_{int}. The shaded background indicates regime: blue for low entropy, orange for high entropy.

a) Held-out signal separation.: On a held-out evaluation, simplified PSAIM exhibits directional separation in all three derived signals: V_{epi} averages 0.025 [0.024, 0.027] in low-entropy blocks and 0.064 [0.062, 0.066] in high-entropy blocks, V_{ale} averages 0.057 [0.057, 0.058] versus 0.072 [0.072, 0.073], and the gate output is 0.930 versus 0.829. These separations are robust across independent reruns of the agent under the same protocol. The mean intrinsic reward is +0.004 in low-entropy blocks and -0.033 in high-entropy blocks in this run, with paired difference $-0.037\,[-0.050, -0.026]$. The paired difference between regimes (low minus high) is consistently negative across independent reruns, so the agent's intrinsic reward is consistently *less negative* in low-entropy regimes than in high-entropy regimes. The *sign* of the low-entropy intrinsic reward, however, is sensitive to the calibration of λ and σ_0^2 against the absolute magnitudes of V_{epi} and V_{ale}, and varies across independently seeded training runs. We therefore claim a robust directional response in r_{int}, not a robust sign change between regimes; numbers reported above are anchored to the original 10-seed evaluation. The directional response is the operationally meaningful property: it is what drives the conservative-admission behaviour observed downstream (Section VII-F), and it is robust across reruns. Sign-level interpretation, when needed, requires a per-deployment calibration of λ and σ_0^2 against the absolute magnitudes of V_{epi} and V_{ale} in the target environment; we identify this as a deployment requirement rather than a property the architecture supplies for free.

b) Why V_{epi} is higher in the high-entropy regime.: The motivating intuition in Section VI-A states that in a low-entropy region with poor characterisation the epistemic term dominates, while in a high-entropy region with good characterisation the aleatoric term dominates. The held-out measurements show V_{epi} higher in the high-entropy blocks; this is not a contradiction with the motivating intuition but a clarification of the operational meaning of the surrogate. V_{epi} is the cross-head variance of the policy-consistent continuation (Equation (2)), and ensemble disagreement responds to two distinct sources: (i) genuine model disagreement about a region the agent has not characterised, and (ii) higher noise in the targets the heads are fitting, which inflates head-wise variance even after long training. The high-entropy regime drives the second mechanism more strongly than the low-entropy regime, and the empirical separation reflects this. The

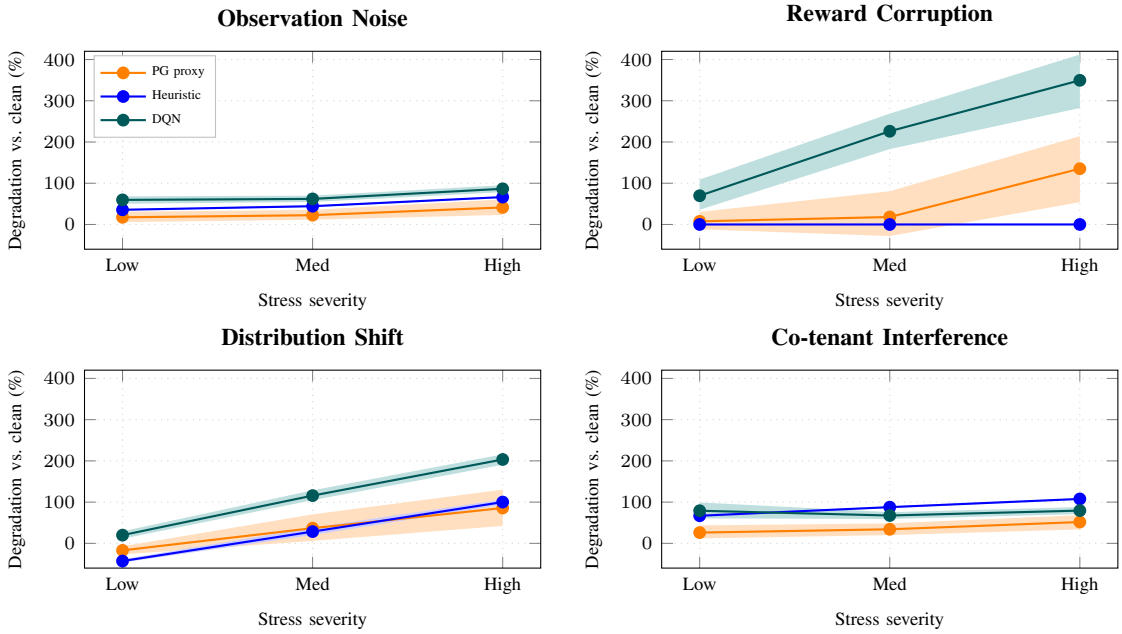

Fig. 4: Robustness degradation across orchestration-like stressors. Each panel shows mean degradation in average slowdown (%) relative to the clean baseline as a function of stress severity, with shaded 95% bootstrap confidence intervals over 10 seeds. Both learned methods degrade substantially under all four perturbation classes at high severity. The SJF heuristic, included for reference, does not consume a learned reward and is therefore insensitive to reward corruption (panel B); under the other three stressors, even the heuristic degrades, but less than either learned method.

gate is designed precisely to handle the resulting ambiguity: when high V_{epi} is accompanied by high V_{ale}, the gated aleatoric excess remains positive and the agent treats the region as noise-dominated rather than information-rich. This is exactly the behaviour observed: the gate output is lower in high-entropy blocks (0.829 versus 0.930), so the aleatoric penalty is less softened there, and the resulting r_{int} becomes negative. The intrinsic reward, not V_{epi} alone, is what implements the surprise-agnostic switch.

c) Epistemic learning within regimes.: Across training checkpoints, V_{epi} on fixed probe states drops from 0.350 at initialisation to 0.008 in the low-entropy regime and to 0.070 in the high-entropy regime. The paired difference is $-0.342\,[-0.342,\ -0.342]$ for low entropy and $-0.280\,[-0.302,\ -0.251]$ for high entropy. The within-regime decrease is the textbook signature of epistemic uncertainty being reduced by experience. The cross-regime gap in the final values reflects the residual disagreement that ensemble heads exhibit on noisier targets, as discussed above.

d) Exploration behaviour shifts between regimes.: Action-distribution evidence shows that exploration behaviour changes between regimes in a direction consistent with the surprise-agnostic switch. The fraction of decisions with positive intrinsic reward is 0.775 in low-entropy blocks versus 0.472 in high-entropy blocks (paired difference $-0.303\,[-0.370,\ -0.247]$). The defer rate is 0.293 versus 0.227 (paired difference $-0.066\,[-0.082,\ -0.055]$) and the reject rate is 0.070 versus 0.140 (paired difference $0.070\,[0.043,$ $0.094]$). The allocate rate itself is not meaningfully separated between regimes (0.637 versus 0.633), so we phrase the behaviour shift as "higher rejection in high-entropy blocks, higher deferral in low-entropy blocks" rather than as a broad change in allocation policy.

TABLE II: Paired comparison of simplified PSAIM against each baseline on stressed average slowdown. Negative differences indicate PSAIM is better; positive indicate worse. "Supported" requires the entire 95% paired bootstrap CI to lie on one side of zero.

Comparison	Δ slowdown [95% CI]	Status
PSAIM vs. SJF heuristic	$+0.891\,[+0.732, +1.030]$	worse (supported)
PSAIM vs. DQN	$-0.651\,[-1.627, -0.001]$	better (supported)
PSAIM vs. DQN+RND	$-0.618\,[-1.867, +0.186]$	directional only
PSAIM vs. PG proxy	$-0.428\,[-1.529, +0.228]$	directional only

D. Downstream Performance

Figure 6 reports clean and stressed average slowdown and task-failure rate for all five methods. Table II reports paired comparisons of simplified PSAIM against the four baselines on the stressed condition.

a) Stress-evaluation results.: On clean data, the SJF heuristic, DQN, and simplified PSAIM are essentially tied at slowdown $\approx$ 1.07, with PG proxy slightly worse and DQN+RND noisier. Under stress, the heuristic remains the strongest method overall (2.412), and all learned methods

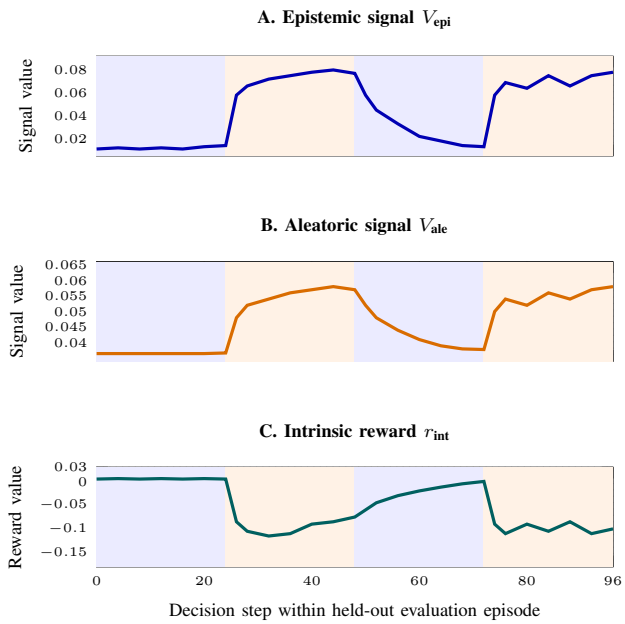

Fig. 5: Simplified PSAIM signals across hidden entropy-regime blocks within a held-out evaluation episode. Background shading indicates regime: blue for low entropy, orange for high entropy. Solid lines show mean signal values; shaded bands show 95% bootstrap confidence intervals. Both V_{epi} (A) and V_{ale} (B) are systematically higher in high-entropy blocks than in low-entropy blocks. The intrinsic reward r_{int} (C) is consistently less negative in low-entropy blocks than in high-entropy blocks; the absolute sign in low-entropy regimes is sensitive to the calibration of λ and σ_0^2 and varies across independent training runs.

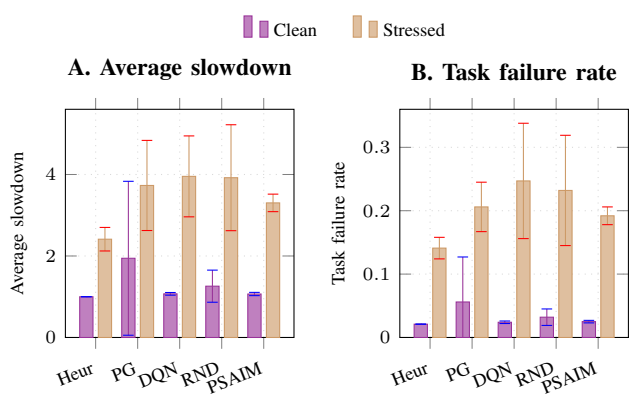

Fig. 6: Downstream performance on clean and stressed evaluation. Bars show means with 95% bootstrap confidence intervals over 10 seeds. The SJF heuristic is the strongest method on both metrics under both conditions. Among learned methods, simplified PSAIM has the lowest stressed slowdown and the second-lowest stressed task-failure rate.

degrade more than the heuristic does. Simplified PSAIM has the lowest stressed slowdown among learned methods (3.303), and the paired comparison against plain DQN is supported (entire bootstrap CI below zero). The improvement of PSAIM over DQN+RND and over the PG proxy is directional but does not pass the strict bootstrap-CI test. We therefore make no claim of broad statistical superiority over all learned baselines.

b) The heuristic gap on the original evaluations is structural.: A targeted diagnostic with longer training and online evaluation adaptation did not close the heuristic gap on the clean and stressed evaluations of Section VII-D (simplified PSAIM 3.46 versus heuristic 2.49 stressed slowdown), suggesting the gap on those evaluations is structural in the current proxy rather than an artefact of training budget. A heuristic that is well matched to the proxy's parametric regularity will, by construction, set a high bar there. The heuristic-breaking analysis of Section VII-F addresses the complementary question: when the proxy's parametric regularity is broken, the same heuristic loses its advantage and a learned policy with a conservative admission prior is the better choice.

E. Ablations

Table III reports stressed slowdown for the simplified PSAIM agent with each major component removed in isolation. Removing the aleatoric penalty hurts substantially (4.461 versus 3.303 for the full agent), confirming that the aleatoric term carries meaningful information in the proxy. Removing the epistemic gate (3.316) or removing reward-object freezing (3.337) does not produce a statistically detectable change against the full agent.

The honest reading of this ablation is twofold. First, in the present proxy, only the aleatoric component does observable work; the gate and the freezing protocol are not load-bearing at this scale. Second, the absence of an effect is not by itself a refutation of the gate's or freezing protocol's design rationale, because the proxy does not exercise the conditions they were designed for. The gate is a guardrail against the noisy-TV failure mode, in which an agent is drawn indefinitely to irreducible noise; the proxy's high-entropy blocks contain stochasticity but no reward-relevant noise source that the agent can be drawn to repeatedly, so the gate's design target is absent. The freezing protocol decouples reward measurement from policy updates over windows long enough for the reward-defining objects to drift. We tested this latter claim directly by repeating the ablation at five times the training horizon and on a monotonically drifting evaluation episode; even at this longer horizon, neither the no-gate variant nor the no-freezing variant differed significantly from the full agent (paired CIs cross zero). We therefore do not claim that the gate or the freezing protocol are vindicated even at $5\times$ horizon in this proxy. The narrow conclusion we draw is that aleatoric penalisation is the component that does the most observable work in this proxy at any horizon we have tested. A focused ablation in environments richer than the proxy, specifically environments with reward-relevant aleatoric distractors and reward-defining objects that drift over operationally meaningful horizons, is required before drawing any positive conclusion about whether

TABLE III: Ablation of simplified PSAIM components on stressed average slowdown, mean with 95% bootstrap CI over 10 seeds. Lower is better.

Configuration	Stressed slowdown [95% CI]
Simplified PSAIM (full)	3.303 [3.103, 3.518]
— no aleatoric penalty	4.461 [3.003, 7.040]
— no epistemic gate	3.316 [3.118, 3.561]
— no reward freezing	3.337 [3.116, 3.593]

the gate and freezing protocol earn their place in the architecture. We identify this as a specific open question.

F. Heuristic-Breaking Distribution Shift

The downstream comparisons in Section VII-D hold the heuristic's structural assumptions intact. SJF best-fit relies on observed job durations being approximately faithful to the underlying duration distribution; once this assumption is violated, the heuristic continues to operate but no longer enjoys the structural advantage that produced its strong showing in earlier sections. We construct a deliberate test of this regime by injecting a systematic bias into the duration field used by the allocator, varying the bias magnitude across three severities (low, medium, high), and re-evaluating SJF and simplified PSAIM on the resulting evaluation distribution. The training distribution and PSAIM hyperparameters are unchanged from the main experiments; only the inference-time observation of duration is perturbed.

Figure 7 reports average slowdown for SJF and simplified PSAIM at the three severities. The heuristic's average slowdown rises from 5.71 to 6.06 as the duration signal becomes increasingly biased; PSAIM's average slowdown is statistically indistinguishable from 3.30 across all three severities and lies entirely below the heuristic at every severity. Table IV summarises the paired comparisons: the difference in average slowdown is $-2.35\,[-2.54, -2.17]$ at low severity, $-2.79\,[-2.92, -2.64]$ at medium severity, and $-2.80\,[-2.95, -2.64]$ at high severity, with all confidence intervals lying entirely below zero by wide margins. The task-failure rate is also lower for PSAIM at every severity (paired difference -0.11 to -0.16, all CIs below zero). We acknowledge that p95 completion time goes the other way: the SJF heuristic completes individual jobs faster in the tail (paired difference $+24$ to $+30$ seconds) because it admits more work per unit time. We do not claim that PSAIM dominates the heuristic on every metric; we claim that on average slowdown and failure rate, in a regime where the heuristic's input assumption is violated, PSAIM is the better policy.

a) Mechanism: a learned conservative admission prior.: Action-distribution evidence clarifies how PSAIM achieves the slowdown advantage. Under all three severities of duration misreporting, simplified PSAIM allocates 82.6–83.5 % of decisions, defers almost never (0.3 %), and rejects 16.2–17.1 %. The SJF heuristic, by contrast, allocates roughly 92 % of decisions and defers the remainder; it never rejects. The paired

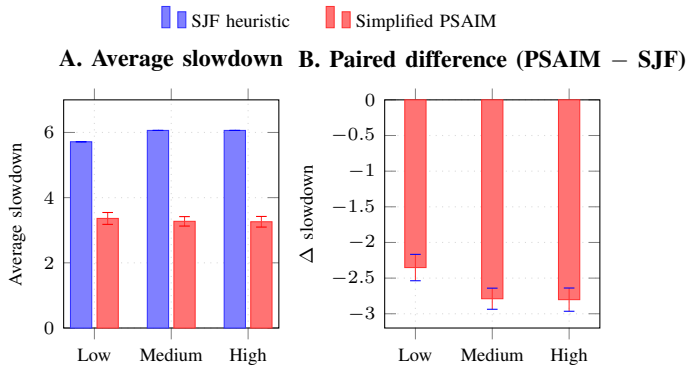

Fig. 7: Heuristic-breaking distribution shift: simplified PSAIM versus SJF best-fit under three severities of duration misreporting. Bars show means; error bars show 95% bootstrap CIs over 10 seeds. Panel B reports the paired-bootstrap difference; all CIs lie entirely below zero, supporting the claim that PSAIM has lower average slowdown than the heuristic at every tested severity.

TABLE IV: Heuristic-breaking distribution shift: paired comparison of simplified PSAIM against SJF best-fit on average slowdown, task-failure rate, and p95 completion time, by severity. Negative differences indicate PSAIM is better; positive differences indicate the heuristic is better. "Supported" requires the entire 95% paired bootstrap CI to lie on one side of zero.

Severity	Metric	Δ [95% CI]	Status
Low	avg. slowdown	$-2.35\,[-2.54, -2.17]$	better (supported)
Low	failure rate	$-0.11\,[-0.13, -0.10]$	better (supported)
Low	p95 completion	$+30.15\,[+27.63, +32.69]$	worse (supported)
Medium	avg. slowdown	$-2.79\,[-2.92, -2.64]$	better (supported)
Medium	failure rate	$-0.16\,[-0.17, -0.15]$	better (supported)
Medium	p95 completion	$+23.82\,[+21.54, +26.16]$	worse (supported)
High	avg. slowdown	$-2.80\,[-2.95, -2.64]$	better (supported)
High	failure rate	$-0.16\,[-0.17, -0.15]$	better (supported)
High	p95 completion	$+23.93\,[+21.59, +26.31]$	worse (supported)

difference in rejection rate against SJF is $+0.16$ to $+0.17$ at every severity, with confidence intervals entirely above zero. PSAIM has converged on a policy that withholds roughly one in six allocations.

The policy is conservative rather than adaptive. On a clean (non-misreport) evaluation under the same training regime, simplified PSAIM rejects 0.182 [0.170, 0.195] of decisions: a rate slightly higher than under any misreport severity. The agent does not respond to corruption by increasing rejection; it has learned a stable conservative admission prior of approximately 17–18 % under regime variation, and that prior happens to be well-suited to perturbations that corrupt heuristic ordering inputs. The mechanism is conservative admission, not corruption detection. The heuristic admits everything and is misled by the corrupted ordering; PSAIM admits less and is

correspondingly less misled. The trade-off is visible in the metrics: PSAIM's lower average slowdown reflects the better ordering of the work it does admit, while the heuristic's lower p95 completion time reflects its higher admission rate.

A static conservative-admission prior, learned without any explicit reward for robustness, can outperform a strong domain heuristic when the heuristic's input assumption is violated. We expect the advantage to extend to other regimes in which conservative admission outperforms aggressive admission, namely those where heuristic ordering inputs are corrupted, but not to regimes where the ordering is reliable: there, the same conservative prior would forgo throughput unnecessarily. This boundary is consistent with the observation in Section VII-D that SJF outperforms PSAIM on the original (clean and stressed) evaluations, where ordering is intact. A more capable agent would scale rejection with detected corruption rather than apply a static prior; the present realisation does not, and we identify this as a concrete direction for follow-up.

b) State-conditional rejection beats fixed-rate rejection.: The natural question raised by the conservative-admission finding is whether a hand-crafted variant of the heuristic that rejects at a comparable marginal rate would suffice to reproduce PSAIM's advantage. We answered this empirically. Define SJF-R(p) as the SJF best-fit heuristic augmented with a uniform random rejection of probability p at each decision; the rejection coin flip is independent of the observed state. We evaluated SJF-R(0.10), SJF-R(0.17), SJF-R(0.18), and SJF-R(0.25) on the same duration-misreport distributions as simplified PSAIM, where 0.17–0.18 matches the rate at which PSAIM rejects in this regime. At all three severities, simplified PSAIM has substantially lower average slowdown than SJF-R(0.17), with paired bootstrap differences of $-1.49\,[-1.73, -1.24]$ at low severity, $-2.03\,[-2.13, -1.91]$ at medium severity, and $-1.89\,[-2.08, -1.70]$ at high severity, all CIs entirely below zero. The failure-rate differences are similarly all below zero. Across the SJF-R sweep from $p = 0.10$ to $p = 0.25$, no fixed rate matches PSAIM's slowdown.

The honest reading is that the marginal rejection rate alone does not produce PSAIM's advantage: PSAIM's policy rejects the right tasks, not just the right fraction of tasks. Whatever conditioning PSAIM has learned on the observed state, it is operationally meaningful at the proxy scale, and a hand-crafted heuristic with a matched marginal rejection rate cannot reproduce it. This rules out the simplest alternative explanation of the heuristic-breaking finding.

c) The explicit decomposition is not load-bearing in this proxy.: A separate ablation tests whether PSAIM's behaviour requires the explicit epistemic-aleatoric decomposition (Equation (2)), or whether a simpler raw total-variance penalty would produce the same operational behaviour. We constructed PSAIM-RawVar, which retains the ensemble of $K = 5$ scalar Q-heads and the same λ and σ_0^2 but replaces the intrinsic reward with $-\lambda \log(1 + \mathrm{Var}(Y_k)/\sigma_0^2)$, with no decomposition into epistemic and aleatoric components and no gate. On the duration-misreport evaluation, PSAIM-RawVar matches simplified PSAIM on average slowdown (paired CI crosses zero at medium and high severities) and exhibits a near-identical rejection rate (16.0–16.6 % versus 16.7–17.8 %). At the proxy scale, the explicit decomposition is therefore not empirically load-bearing for the duration-misreport advantage. The decomposition retains theoretical motivation (the gate's analytical role and the noisy-TV guardrail; Section VI-C) but its empirical necessity is an open question for environments richer than the present proxy.

d) Summary of empirical findings.: The four stress scenarios reproduce, in a controlled proxy, the qualitative degradation pattern predicted by the adversarial-ML reading of Section IV-B; both learned baselines fail in the predicted ways, and the matched-load control isolates stochasticity as the operative source of harm in the co-tenant case. The PSAIM realisation exhibits directional separation in V_{epi}, V_{ale}, and gate values across hidden entropy regimes, with r_{int} consistently less negative in low-entropy regimes than in high-entropy regimes; we phrase this as a directional response rather than a robust sign switch given seed-level fragility. On the original (clean and stressed) downstream evaluations, simplified PSAIM improves on plain DQN under stress and remains below the SJF heuristic; the heuristic gap on these evaluations is structural in this proxy and not an undertraining artefact. On a heuristic-breaking distribution shift, simplified PSAIM has lower average slowdown and lower failure rate than SJF at every tested severity, and beats a hand-crafted SJF + uniform-random-rejection control with the same marginal rejection rate by paired margins entirely below zero, indicating that the rejection conditioning (not just the rate) is operationally meaningful. A separate ablation against a raw-total-variance penalty matches simplified PSAIM at the proxy scale, indicating that the explicit epistemic-aleatoric decomposition is not empirically load-bearing for the duration-misreport advantage in this proxy and remains an open question for richer environments. These results support the mechanism-level claims of Sections V-A and VI and provide one operational regime in which a D1 realisation outperforms the strongest baseline available; they are not evidence of deployment-readiness, the scope of which is discussed in Section IX.

VIII. Related Work

a) RL for systems.: Deep RL for cluster resource management and scheduling was advanced by Mao *et al.* [4], [3], with antecedents in hybrid autonomic approaches [5]. Mao *et al.* [24] subsequently released the Park benchmark, an open platform that aggregates twelve systems-centric optimisation problems under a common RL interface and explicitly catalogues the ways in which RL for systems differs from RL for games; the benchmark is the closest existing analogue to the position we develop here, and we read its motivating discussion as evidence that the orchestration-RL community itself has identified the structural mismatch we elaborate. Luong *et al.* [6] survey deep RL applications across networking and communications; Rossi *et al.* [2] apply RL to container scaling. The shared finding across this literature

is that RL can outperform heuristics in simulation but has not displaced production heuristics such as those surveyed by Lorido-Botran *et al.* [20] and Qu *et al.* [21], even when the deployment infrastructure (Park, OpenAI Gym wrappers, RLlib) is in place. This is not for lack of available frameworks; it is a substantive gap that the present paper attributes to environment structure rather than tooling.

b) Learning-augmented systems as a complementary direction.: A separate line of work avoids end-to-end RL altogether and instead augments classical algorithms with learned predictions, an approach often called learning-augmented algorithms or, more broadly, learning-augmented systems [24]. The argument we make in this paper is orthogonal to that direction. Learning-augmented systems address a different question: assuming a classical algorithm is the right structural choice, how can learning improve its inputs (predictions, parameter selection, workload forecasts)? The directions identified in Section V address a complementary question: when learning is used as the policy itself, what should the learning component look like so that its assumptions match orchestration environments? A complete future system may well combine the two: a learning-augmented classical algorithm whose learned-prediction component is itself uncertainty-decomposed in the sense of D1, action-space-typed in the sense of D2, and perturbation-aware in the sense of D3. We do not see the choice between learning-augmented systems and direct learned policies as binary, and we read the persistent practitioner preference for the former [7] as further evidence that the structural concerns raised in this paper are taken seriously even when the response is to keep learning at one remove from the policy itself.

c) Where RL has succeeded in adjacent systems contexts.: The argument that RL has not displaced production heuristics is specific to orchestration; it is not a general claim about RL in systems. Adaptive bitrate streaming illustrates one adjacent context where learned policies have been productive: Mao *et al.* [45] demonstrate that an RL-trained ABR controller can outperform hand-tuned models without relying on hand-tuned bitrate-quality models. Compared with orchestration, ABR streaming operates per-client, with reversible decisions on millisecond timescales and a localised reward (the user's quality of experience) that is not contaminated by other users' traffic. It exhibits neither the cluster heterogeneity nor the multi-tenant interference, proxy-reward composition, or non-stationary regime structure that we identify as the operative obstacles for orchestration RL. The analytical framework of Section IV therefore both predicts and is consistent with the success of RL in adjacent contexts and its absence in production orchestration: where the structural mismatch is mild or absent, RL reaches deployment; where the mismatch is pervasive, it does not.

d) Intrinsic motivation.: The intrinsic-motivation literature spans prediction-error [14], [15], disagreement [44], learning-progress [33], count-based [34], and information-gain [35] objectives; Badia *et al.* [37] combine strategies via a meta-controller. D1 differs from all of these by advocating an explicit decomposition rather than a new bonus or a meta-controller.

e) Uncertainty decomposition and information-directed sampling.: Depeweg *et al.* [38] established the epistemic-aleatoric decomposition for risk-sensitive RL with Bayesian networks, with related ideas in computer vision [39]. Osband *et al.* [43] used ensemble disagreement as an exploration bonus. Russo and Van Roy [40] introduced information-directed sampling, which trades off information gain against regret in the bandit and tabular-MDP settings; the PSAIM objective in Equation (1) is analogous in spirit, though it targets the deep-RL setting with function approximation via a model-free surrogate. The PSAIM instantiation is consistent with this lineage and extends it by combining the decomposition with a short-horizon gate and a block-wise stationarity protocol.

f) Adversarial robustness of RL.: Huang *et al.* [28], Gleave *et al.* [30], and Zhang *et al.* [32] study adversarial attacks on RL; Rakhsha *et al.* [29] study reward poisoning; Behzadan and Munir [31] study policy-induction attacks. The contribution of Section IV-B is not a new attack or defence but a reframing: natural orchestration phenomena produce the same structural effects as deliberate attacks, a correspondence that is not drawn explicitly in the existing literature.

g) Robust MDPs and the systems-uncertainty perspective.: The mathematically established framework for non-adversarial uncertainty in transitions, observations, and rewards is the Robust MDP literature: Iyengar [46] and Nilim and El Ghaoui [47] introduce the rectangular uncertainty-set formulation that supports tractable robust dynamic programming, and Wiesemann *et al.* [48] extend the analysis to general uncertainty sets. Pinto *et al.* [49] bring the formulation into deep RL with adversarial training of an auxiliary disturbance policy. Our adversarial-ML reading and the Robust MDP framing are complementary rather than competing. The Robust MDP literature provides the formal language for bounded uncertainty sets and minimax policy guarantees; the adversarial-ML reading provides a different rhetorical lens that emphasises the structural correspondence between natural orchestration phenomena and known fragility modes of deep RL. We use the latter framing because the orchestration community is more likely to recognise the failure modes than the formalism, and because the empirical literature on adversarial RL fragility is more directly applicable to the deep-value-function methods that current orchestration RL papers actually use. A complete treatment of the directions in Section V should eventually engage both: D3 in particular is naturally specifiable as a robust-MDP training procedure with uncertainty sets calibrated to realistic telemetry-noise and tenant-mixture distributions.

IX. Discussion and Limitations

a) Scope of the contribution.: This is a position paper. Its contribution is not a deployed system but a reframing of the problem and a programme for what should come next. It offers a taxonomy of orchestration tasks by their MDP properties, a structural analysis that reads natural orchestration phenomena as adversarial-ML attacks in disguise, three research directions

designed around environment structure, and a mechanism realisation of one direction with empirical evidence at the proxy scale. It does not present a deployed system, a production-scale evaluation, or full-stack comparisons against operational orchestrators; the empirical programme is mechanism-level evidence in service of the position, not a competitive systems result.

The taxonomy in Section III treats task classes independently and is used in Section V to align research directions with the classes they most directly address. Real orchestration systems involve cross-class interactions that a cross-class taxonomy would expose; we have adopted the independent treatment because it is sufficient for the per-class predictions the paper makes and because a cross-class taxonomy would require empirical grounding beyond what a position paper can provide.

The adversarial-ML framing in Section IV-B is structural rather than worst-case. Natural noise is not adversarial in magnitude, and the bounds from the adversarial literature may overstate fragility in practice. The stress experiments in Section VII-B calibrate perturbation magnitudes to realistic telemetry noise rather than worst-case bounds, and the degradations they show should be read as evidence of fragility under realistic conditions rather than as quantitative predictions for production.

The empirical findings were performed in a single-node, single-agent proxy. The proxy is sufficient for mechanism-level claims (does a perturbation degrade the policy in the predicted direction; does PSAIM's intrinsic reward switch sign in the predicted direction) but not for deployment-level claims. The SJF heuristic is the strongest method on the proxy under both clean and stressed conditions, and longer training does not close this gap. We interpret this as the expected outcome of a controlled environment in which the parametric structure that motivates a strong heuristic is preserved while the structural properties that motivate learning are simulated under tight bounds; Section VII-F addresses the complementary regime in which that parametric regularity is broken.

The PSAIM realisation in Section VI has three known mechanism gaps that the empirical programme exposes. First, several ingredients required for formal guarantees (calibration of ensemble heads, bounded aleatoric estimators, stationarity protocols) are described as part of the full mechanism but are not stress-tested here. Second, there is no reward-trust mechanism: the realisation defends against state-transition uncertainty but not against direct reward-channel corruption, which is why low-severity reward corruption is partially absorbed while high-severity corruption is not. A reward prediction ensemble or downweighting of extrinsic reward when corruption indicators are detected would extend the realisation in this direction. Third, the explicit epistemic-aleatoric decomposition that motivates the architecture (Equation (2)) is matched at the proxy scale by a simpler raw-total-variance penalty (Section VII-F); the decomposition's empirical necessity is therefore an open question for richer environments where the gate's analytical role becomes operationally relevant. We identify these as concrete extensions for follow-up work.

The research directions in Section V are not mutually exclusive: a learning system that instantiates D1, D2, and D3 simultaneously is conceivable and arguably desirable. We have presented them as separate directions to make the individual arguments falsifiable. Nothing in this paper argues that unsupervised learning signals alone are sufficient for orchestration: in deployed systems, extrinsic objectives aligned with operator intent will remain central, and the directions we advocate are complementary in that they produce agents whose learning assumptions are less violated by orchestration environments.

b) A deployment-evaluation roadmap.: A definitive test of the directions identified here would require four elements that the present paper does not deliver. First, a multi-tenant cluster simulator or testbed that captures network and storage interference at task granularity, rather than the single-node proxy used here. Second, a workload corpus spanning at least three distinct entropy profiles drawn from production traces such as those of Reiss *et al.* [26] or Cortez *et al.* [27], so that cross-cluster generalisation can be tested directly. Third, baselines drawn from both the RL-for-systems community (Decima [3], Park [24]) and the systems community (priority-based bin-packing in the Borg style [8], [9], default Kubernetes scalers, Autopilot's recommenders [7]), so that comparisons span both the academic and operational state of the art. Fourth, per-class evaluation aligned with Table I, so that the predictions D1 and D2 make for different orchestration task classes are tested where the predictions actually differ. We further argue, as a normative claim about how the field should be evaluated, that orchestration RL papers should be expected to include perturbation evaluation as a default rather than as an optional robustness check; the long-standing convention in supervised learning of reporting test-set performance has its analogue here in reporting performance under realistic telemetry noise, reward proxy bias, and tenant-mixture shift. The absence of such reporting in current practice is part of what the present paper aims to change.

X. Conclusion

We have argued that the persistent gap between RL research on service orchestration and RL adoption in production orchestrators is a predictable consequence of applying standard RL formulations to environments whose structural properties violate standard RL assumptions. The taxonomy in Section III organises orchestration tasks by the MDP properties that govern learnability; the analysis in Section IV-B establishes a structural correspondence between natural orchestration phenomena and adversarial-machine-learning attack classes; the directions in Section V reorient the design of learning methods around environment structure; and PSAIM in Section VI demonstrates that one such direction admits a mechanism realisation whose behaviour is empirically observable. The realisation produces, in a single-node proxy and under a heuristic-breaking distribution shift, an operational regime in which a D1-aligned policy outperforms a strong domain heuristic. The mechanism is not adaptive corruption detection: it is a

state-conditional rejection policy with a marginal rate of approximately 17–18 % that the agent has learned under regime variation. A hand-crafted heuristic that rejects at the same marginal rate but uniformly at random performs significantly worse than PSAIM across all tested severities, indicating that the conditioning matters and is not reproducible by a simpler fixed-rate alternative. We expect this advantage to extend to other regimes in which conservative admission outperforms aggressive admission, namely those where heuristic ordering inputs are corrupted, but not to regimes in which the ordering is reliable; the same conservative prior would forgo throughput unnecessarily in the latter case. We identify direct empirical tests of this boundary as concrete follow-up.

If the community accepts this framing, several consequences follow. The analysis suggests that the locus of inductive bias for orchestration RL should shift from state and reward to action structure (D2), changing which architectural choices require justification in future work. Empirical evaluation under perturbation becomes a default rather than an optional robustness check (D3), changing which results count as informative about deployability. Unsupervised learning signals with explicit uncertainty decomposition become a viable alternative to portfolio-of-strategies meta-controllers (D1), changing which baselines a new orchestration method must compare against. Production orchestrators that deploy learning-based components, such as Autopilot's supervised recommenders, become evidence that learning is not the obstacle; the obstacle is the specific learning paradigm that has dominated academic attention.

a) A view of the next five years.: If the community accepts the framing of this paper, the orchestrators of the late 2020s should not look like end-to-end RL pipelines that ship to production. They should look like hybrid systems in which classical control and queueing logic remains responsible for the act of deciding, while learned components produce the predictions, the priors, and the uncertainty estimates that the deciding logic conditions on. The locus of learning should move from policy to perception. Production-grade orchestrators should treat their learning components the way safety-critical systems treat their sensors: as fallible inputs whose failure modes are understood, instrumented, and bounded. Within this picture, D1 contributes uncertainty-aware perception that distinguishes what an agent does not know from what it cannot know; D2 contributes the structural commitment that decisions live in well-typed action spaces with semantics the operator can reason about; D3 contributes the evaluation discipline that filters out methods whose nominal performance does not survive realistic perturbation. This is broadly consistent with the trajectory the distributed-systems community has begun to articulate for the computing continuum [50], in which adaptation across heterogeneous and partially observable infrastructure is treated as a first-class concern rather than as an engineering afterthought. The work this paper proposes is one piece of the technical content that such a trajectory will require.

What this paper does not provide is the demonstration that any of D1, D2, or D3, individually or jointly, produces a deployment-ready orchestration policy. That demonstration requires environments richer than the proxy reported here, comparisons against systems-community baselines that the present work does not include, and engineering investment that a position paper cannot supply on its own. The PSAIM realisation also has known mechanism gaps: the absence of a reward-trust mechanism that would extend its robustness from low-severity reward corruption to severe reward-channel attacks, and the observation that while PSAIM's rejection conditioning beats a uniform-random control of matched marginal rate, the marginal rejection rate itself does not vary with corruption severity, which limits the strength of any adaptive-detection claim. We identify these as specific extensions rather than as defences.

The argument of this paper, reduced to one sentence, is that the next decade of orchestration research will be more productive if it asks what kinds of learning are fit for orchestration than if it continues to ask how orchestration can be fit to standard RL. The two questions are not equivalent, and the choice of which one structures the field will determine what gets built.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[2] F. Rossi, M. Nardelli, and V. Cardellini, "Horizontal and vertical scaling of container-based applications using reinforcement learning," in *Proceedings of the IEEE 12th International Conference on Cloud Computing (CLOUD)*. IEEE, 2019, pp. 329–338.

[3] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proceedings of the ACM SIGCOMM 2019 Conference*. ACM, 2019, pp. 270–288.

[4] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*. ACM, 2016, pp. 50–56.

[5] G. Tesauro, N. K. Jong, R. Das, and M. N. Bennani, "A hybrid reinforcement learning approach to autonomic resource allocation," in *Proceedings of the 3rd IEEE International Conference on Autonomic Computing (ICAC)*. IEEE, 2006, pp. 65–73.

[6] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang, and D. I. Kim, "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Communications Surveys and Tutorials*, vol. 21, no. 4, pp. 3133–3174, 2019.

[7] K. Rzadca, P. Findeisen, J. Swiderski, P. Zych, P. Broniek, J. Kusmierek, P. Nowak, B. Strack, P. Witusowski, S. Hand, and J. Wilkes, "Autopilot: Workload autoscaling at Google," in *Proceedings of the 15th European Conference on Computer Systems (EuroSys)*. ACM, 2020, pp. 1–16.

[8] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proceedings of the 10th European Conference on Computer Systems (EuroSys)*. ACM, 2015, pp. 1–17.

[9] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.

[10] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[11] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100.

[12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[13] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR, 2018, pp. 1861–1870.

[14] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*. PMLR, 2017, pp. 2778–2787.

[15] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Exploration by random network distillation," in *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019.

[16] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, "Multi-resource packing for cluster schedulers," in *Proceedings of the ACM SIGCOMM 2014 Conference*. ACM, 2014, pp. 455–466.

[17] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *Journal of Machine Learning Research*, vol. 16, no. 1, pp. 1437–1480, 2015.

[18] C. Delimitrou and C. Kozyrakis, "Quasar: Resource-efficient and QoS-aware cluster management," in *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2014, pp. 127–144.

[19] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zanis, A. Vizzini, D. Petrovska, A. Phanishayee, and C. Delimitrou, "An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2019, pp. 3–18.

[20] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, 2014.

[21] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: A taxonomy and survey," *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–33, 2018.

[22] C. J. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[23] S. Padakandla, "A survey of reinforcement learning algorithms for dynamically varying environments," *ACM Computing Surveys*, vol. 54, no. 6, pp. 1–25, 2021.

[24] H. Mao, P. Negi, A. Narayan, H. Wang, J. Yang, H. Wang, R. Marcus, R. Addanki, M. Khani Shirkoohi, S. He, V. Nathan, F. Cangialosi, S. B. Venkatakrishnan, W.-H. Weng, S. Han, T. Kraska, and M. Alizadeh, "Park: An open platform for learning-augmented computer systems," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 2490–2502.

[25] W. Zhao, J. P. n. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," *arXiv preprint arXiv:2009.13303*, 2020.

[26] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, "Heterogeneity and dynamicity of clouds at scale: Google trace analysis," in *Proceedings of the 3rd ACM Symposium on Cloud Computing (SoCC)*. ACM, 2012.

[27] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms," in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*. ACM, 2017, pp. 153–167.

[28] S. Huang, N. Papernot, I. Goodfellow, Y. Duan, and P. Abbeel, "Adversarial attacks on neural network policies," in *ICLR 2017 Workshop Track*, 2017.

[29] A. Rakhsha, G. Radanovic, R. Devidze, X. Zhu, and A. Singla, "Policy teaching via environment poisoning: Training-time attacks on reinforcement learning," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*. PMLR, 2020, pp. 7974–7984.

[30] A. Gleave, M. Dennis, C. Wild, N. Kant, S. Levine, and S. Russell, "Adversarial policies: Attacking deep reinforcement learning," in *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020.

[31] V. Behzadan and A. Munir, "Vulnerability of deep reinforcement learning to policy induction attacks," in *Machine Learning and Data Mining in Pattern Recognition: 13th International Conference (MLDM)*, ser. Lecture Notes in Computer Science, vol. 10358. Springer, 2017, pp. 262–275.

[32] H. Zhang, H. Chen, C. Xiao, B. Li, M. Liu, D. Boning, and C.-J. Hsieh, "Robust deep reinforcement learning against adversarial perturbations on state observations," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020, pp. 21 024–21 037.

[33] J. Schmidhuber, "Formal theory of creativity, fun, and intrinsic motivation (1990–2010)," *IEEE Transactions on Autonomous Mental Development*, vol. 2, no. 3, pp. 230–247, 2010.

[34] M. G. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos, "Unifying count-based exploration and intrinsic motivation," in *Advances in Neural Information Processing Systems 29 (NeurIPS)*, 2016, pp. 1471–1479.

[35] R. Houthooft, X. Chen, Y. Duan, J. Schulman, F. De Turck, and P. Abbeel, "VIME: Variational information maximizing exploration," in *Advances in Neural Information Processing Systems 29 (NeurIPS)*, 2016, pp. 1109–1117.

[36] Y. Burda, H. Edwards, D. Pathak, A. Storkey, T. Darrell, and A. A. Efros, "Large-scale study of curiosity-driven learning," in *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019.

[37] A. P. Badia, B. Piot, S. Kapturowski, P. Sprechmann, A. Viber, Z. D. Guo, and C. Blundell, "Agent57: Outperforming the Atari human benchmark," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*. PMLR, 2020, pp. 507–517.

[38] S. Depeweg, J. M. Hernández-Lobato, F. Doshi-Velez, and S. Udluft, "Decomposition of uncertainty in Bayesian deep learning for efficient and risk-sensitive learning," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR, 2018, pp. 1184–1193.

[39] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?" in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5574–5584.

[40] D. Russo and B. Van Roy, "Learning to optimize via information-directed sampling," in *Advances in Neural Information Processing Systems 27 (NeurIPS)*, 2014, pp. 1583–1591.

[41] W. Dabney, M. Rowland, M. G. Bellemare, and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018, pp. 2892–2901.

[42] M. G. Bellemare, W. Dabney, and R. Munos, "A distributional perspective on reinforcement learning," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*. PMLR, 2017, pp. 449–458.

[43] I. Osband, C. Blundell, A. Pritzel, and B. Van Roy, "Deep exploration via bootstrapped DQN," in *Advances in Neural Information Processing Systems 29 (NeurIPS)*, 2016, pp. 4026–4034.

[44] D. Pathak, D. Gandhi, and A. Gupta, "Self-supervised exploration via disagreement," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*. PMLR, 2019, pp. 5062–5071.

[45] H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with Pensieve," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 2017, pp. 197–210.

[46] G. N. Iyengar, "Robust dynamic programming," *Mathematics of Operations Research*, vol. 30, no. 2, pp. 257–280, 2005.

[47] A. Nilim and L. El Ghaoui, "Robust control of Markov decision processes with uncertain transition matrices," *Operations Research*, vol. 53, no. 5, pp. 780–798, 2005.

[48] W. Wiesemann, D. Kuhn, and B. Rustem, "Robust Markov decision processes," *Mathematics of Operations Research*, vol. 38, no. 1, pp. 153–183, 2013.

[49] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, "Robust adversarial reinforcement learning," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017, pp. 2817–2826.

[50] S. Dustdar, V. Casamayor Pujol, and P. K. Donta, "On distributed computing continuum systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 4, pp. 4092–4105, 2023.